

APUNTES

Aquí tienes un resumen ampliado de comandos básicos y avanzados de Linux, incluyendo algunos más complejos como la actualización del sistema, la clonación de repositorios, descargas y otros útiles:

Navegación y gestión de directorios

pwd: Muestra la ruta del directorio actual. `pwd`
ls: Lista los archivos y directorios en el directorio actual. `Ls` `ls -l` # Lista en formato detallado `ls -a` # Muestra archivos ocultos
cd: Cambia de directorio. `cd /ruta/al/directorio` # Ir a una ruta específica `cd ..` # Subir un nivel `cd ~` # Ir al directorio home
mkdir: Crea un nuevo directorio. `mkdir nombre_directorio`
rmdir: Elimina un directorio vacío. `rmdir nombre_directorio`

Gestión de archivos

touch: Crea un archivo vacío. `touch archivo.txt`
cp: Copia archivos o directorios. `cp archivo.txt /ruta/destino` `cp -r directorio /ruta/destino` # Copia recursiva para directorios
mv: Mueve o renombra archivos o directorios. `mv archivo.txt /ruta/destino` `mv archivo.txt nuevo_nombre.txt` # Renombrar
rm: Elimina archivos o directorios. `rm archivo.txt` `rm -r directorio` # Elimina recursivamente
cat: Muestra el contenido de un archivo. `cat archivo.txt`
more / less: Muestra el contenido de un archivo página por página. `more archivo.txt` `less archivo.txt`
head / tail: Muestra las primeras o últimas líneas de un archivo. `head archivo.txt` `tail archivo.txt`

Búsqueda y manipulación de texto

grep: Busca texto en archivos. `grep "texto" archivo.txt`
find: Busca archivos o directorios. `find /ruta -name "archivo.txt"`
echo: Muestra texto en la terminal. `echo "Hola, mundo"`
sed: Edita texto en archivos o streams. `sed 's/foo/bar/g' archivo.txt` # Reemplaza "foo" por "bar"
awk: Procesa y analiza texto. `awk '{print $1}' archivo.txt` # Imprime la primera columna

Permisos y usuarios

chmod: Cambia los permisos de un archivo o directorio. `chmod 755 archivo.txt` # Permisos rwxr-xr-x
chown: Cambia el propietario de un archivo o directorio. `chown usuario:grupo archivo.txt`
sudo: Ejecuta un comando como superusuario (root). `sudo comando`

Red y procesos

ping: Prueba la conectividad con un servidor. `ping google.com`
ifconfig / ip: Muestra información de red. `Ifconfig` `ip addr show`
ps: Muestra los procesos en ejecución. `ps aux`
kill: Termina un proceso. `kill PID` # Reemplaza PID con el ID del proceso
top / htop: Muestra los procesos en tiempo real. `Top` `htop`

Compresión y descompresión

tar: Crea o extrae archivos comprimidos. `tar -cvf archivo.tar directorio` # Crear `tar -xvf archivo.tar` # Extraer
gzip / gunzip: Comprime o descomprime archivos. `gzip archivo.txt` `gunzip archivo.txt.gz`
zip / unzip: Comprime o descomprime archivos en formato ZIP. `zip archivo.zip archivo.txt` `unzip archivo.zip`

Actualización del sistema

apt-get: Actualiza el sistema y los paquetes (en distribuciones basadas en Debian/Ubuntu).
`sudo apt-get update` # Actualiza la lista de paquetes
`sudo apt-get upgrade` # Actualiza los paquetes instalados
`sudo apt-get dist-upgrade` # Actualiza el sistema completo
yum: Actualiza el sistema en distribuciones basadas en Red Hat/CentOS. `sudo yum update`
`apt update` # Actualiza la lista de paquetes
`apt upgrade` # Actualiza los paquetes instalados
`apt install nginx` # Instala un paquete
`apt remove nginx` # Elimina un paquete

Gestión de repositorios Git

git clone: Clona un repositorio de Git. `git clone https://github.com/usuario/repositorio.git`
git pull: Actualiza un repositorio local con los cambios remotos. `git pull origin main`
git push: Sube cambios locales al repositorio remoto. `git push origin main`

Descargas desde la terminal

wget: Descarga archivos desde la web. `wget https://ejemplo.com/archivo.zip`
curl: Realiza solicitudes HTTP y descarga archivos. `curl -O https://ejemplo.com/archivo.zip`

Otros útiles

man: Muestra el manual de un comando. `man ls`
history: Muestra el historial de comandos ejecutados. `history`
clear: Limpia la terminal. `clear`
scp: Copia archivos entre máquinas de forma segura (SSH). `scp archivo.txt usuario@servidor:/ruta/destino`
rsync: Sincroniza archivos y directorios entre sistemas. `rsync -avz /ruta/origen/ usuario@servidor:/ruta/destino/`
cron: Programa tareas recurrentes.
Editar el archivo crontab: `crontab -e`
Ejemplo de tarea programada (ejecutar un script cada día a las 2 AM): `0 2 * * * /ruta/al/script.sh`

📌 Guía Rápida: RGPD y el uso de servicios Cloud

♦ 1. Roles según el RGPD

Rol	Quién es	Ejemplo
Titular de los datos	Persona física dueña de sus datos	Un cliente que da su email en una tienda online
Responsable	Decide cómo y por qué se usan los datos	La empresa que usa esos datos
Encargado	Procesa datos por cuenta del responsable	Un proveedor cloud como AWS, Google Cloud, etc.

♦ 2. ¿Quién es el responsable legal?

La empresa siempre es responsable legal de los datos, aunque contrate servicios en la nube o terceros.

3. En caso de incidente (brecha de datos)

La empresa (responsable) debe: Notificar a la AEPD en 72h. Informar a los afectados si es grave. Investigar y mitigar el daño.

El proveedor (encargado): Debe informar inmediatamente al responsable. Puede ser sancionado si incumple sus obligaciones.

Pero la empresa sigue siendo la cara visible y legal ante el RGPD.

Tabla de Permisos chmod

Código	Lectura (r)	Escritura (w)	Ejecución (x)	Significado
0	✗	✗	✗	Sin permisos
1	✗	✗	✓	Solo ejecución
2	✗	✓	✗	Solo escritura
3	✗	✓	✓	Escritura + ejecución
4	✓	✗	✗	Solo lectura
5	✓	✗	✓	Lectura + ejecución
6	✓	✓	✗	Lectura + escritura
7	✓	✓	✓	Todos los permisos (rwx)

Posición de los dígitos en chmod

Posición	Usuario	Ejemplo en chmod 755	Significado
1º	Propietario (u)	7	Todos los permisos (rwx)
2º	Grupo (g)	5	Lectura + ejecución (r-x)
3º	Otros (o)	5	Lectura + ejecución (r-x)

Sigla	Nombre completo	Puerto(s) predeterminado(s)	Cifrado	Protocolo (TCP/UDP)	Vulnerabilidades comunes
HTTP	HyperText Transfer Protocol	80	No	TCP	Tráfico no cifrado, susceptible a sniffing y MITM
HTTPS	HTTP Secure	443	Sí	TCP	Malas configuraciones, versiones antiguas TLS
FTP	File Transfer Protocol	20 (datos), 21 (control)	No	TCP	Transmite credenciales en texto plano, susceptible a sniffing
SFTP	SSH File Transfer Protocol	22	Sí	TCP	Depende de la seguridad de SSH
SSH	Secure Shell	22	Sí	TCP	Fuerza bruta, claves mal protegidas
TELNET	Terminal Network	23	No	TCP	Transmisión en texto plano, muy inseguro
SMTP	Simple Mail Transfer Protocol	25	No	TCP	Relaying, spoofing si no se usa STARTTLS
DNS	Domain Name System	53	No	TCP/UDP	DNS spoofing, cache poisoning
DHCP	Dynamic Host Configuration Protocol	67 (servidor), 68 (cliente)	No	UDP	DHCP spoofing, ataques de denegación de IP
TFTP	Trivial File Transfer Protocol	69	No	UDP	Sin autenticación, vulnerable a ataques MITM
POP3	Post Office Protocol v3	110	No	TCP	Credenciales en texto plano
IMAP	Internet Message Access Protocol	143	Opcional	TCP	Acceso no cifrado, susceptible a sniffing
SNMP	Simple Network Management Protocol	161 (consulta), 162 (traps)	Opcional	UDP	Filtración de red, ataques DoS, exposición de SNMPv1
LDAP	Lightweight Directory Access Protocol	389 (sin cifrar), 636 (con SSL)	Opcional	TCP	Inyección LDAP, acceso sin cifrar
RDP	Remote Desktop Protocol	3389	Sí	TCP	Fuerza bruta, exploits de servicios RDP
SMB	Server Message Block	445	No	TCP	Ejecuta comandos remotamente si está mal configurado
NTP	Network Time Protocol	123	No	UDP	Manipulación del reloj del sistema, DoS
MySQL	MySQL Database Protocol	3306	Opcional	TCP	Inyecciones SQL, credenciales débiles
PostgreSQL	PostgreSQL Database Protocol	5432	Sí	TCP	Inyecciones SQL, configuración incorrecta
RADIUS	Remote Authentication Dial-In User Service	1812 (autenticación), 1813 (contabilidad)	Opcional	UDP	Fuerza bruta, spoofing, configuración insegura
Elasticsearch	Elasticsearch REST API	9200 (HTTP), 9300 (transporte)	Opcional	TCP	Exposición de datos si no está autenticado, ataques a APIs

Inyección SQL:

Inyección	Tipo	¿Válido?	Comentario
' OR 1=1 --	Númerica	✓	Muy común y simple
' OR '1'='1' --	Cadena	✓	Mejor si los valores son strings
' OR 'a'='a' --	Cadena	✓	Idéntico a la anterior

Comparativa del código PHP (Low vs Impossible) injection SQL

Aspecto	Modo Low	Modo Impossible	Explicación breve
Entrada directa	Sí	No	Se usa el dato del usuario directamente en la consulta SQL, lo que es inseguro.
Validación de tipo numérico	No	Sí (is_numeric)	Se comprueba si la entrada es un número antes de ejecutar la consulta.
Consultas preparadas	No	Sí (\$db->prepare)	Separan los datos del código SQL, previniendo inyecciones.
ORM (Object-Relational Mapping)	No	Opcional/implícito	Capa que genera consultas preparadas automáticamente al trabajar con objetos.
Tokens CSRF	No	Sí	Previenen solicitudes maliciosas de formularios desde otros sitios.
Tokens de sesión	No	Sí	Protegen contra secuestro de sesión y aseguran la autenticidad del usuario.
Mensajes de error al usuario	Sí (expuestos)	No (silencio total)	Modo Low muestra errores detallados, lo que ayuda al atacante.

Resultados limitados	No	Sí (por diseño seguro)	En modo seguro se limita la cantidad de datos devueltos para evitar fugas masivas.
Satinización de entrada	No	Si	Filtra el valor antes de usarlo, eliminando caracteres peligrosos

♦ Preguntas y respuestas finales (con correcciones)	Comportamiento del aplicativo web	
Entrada	Modo Low	Modo Impossible
"	Lanza error, se muestra mucha info al usuario	No se ejecuta, no se muestra nada
' OR '1'='1	Devuelve todos los usuarios	No se ejecuta, no se muestra nada

Vulnerabilidades web			
Aspecto	XSS Reflejado	XSS Almacenado	Inyección SQL
Cómo funciona	El código malicioso se refleja desde una entrada del usuario (URL, formulario) en la respuesta del servidor. No se almacena en servidor permanentemente.	El código malicioso se almacena en el servidor (base de datos, foros, etc.) y se ejecuta cuando otros usuarios acceden a la página.	El código SQL malicioso se inyecta en una consulta SQL a través de la entrada del usuario, lo que puede permitir al atacante manipular la base de datos.
Tipo de ataque	Ataque del lado cliente.	Ataque del lado cliente.	Ataque del lado servidor.
Impacto	Requiere que la víctima haga clic en un enlace malicioso. El impacto es inmediato, pero puede ser limitado.	Afecta a todos los usuarios que visitan la página comprometida. El impacto puede ser mayor y persistente.	Puede permitir al atacante leer, modificar o eliminar datos de la base de datos, o incluso ejecutar comandos en el servidor.
Mitigaciones	- Codificación de salida (servidor). - Validación y sanitización de entrada (servidor). - Política de seguridad de contenido (CSP) (servidor, el navegador la interpreta). - Escapado de contenido HTML (servidor). - Actualización del software (servidor). - Configuración de cookies (Atributos: HttpOnly y Secure) (servidor).	- Codificación de salida (servidor). - Validación y sanitización de entrada (servidor). - Política de seguridad de contenido (CSP) (servidor, el navegador la interpreta). - Escapado de contenido HTML (servidor). - Actualización del software (servidor). - Atributos de cookies (HttpOnly y Secure) (servidor).	- Limitar resultados - Tokens de sesión o CSRF - Sentencias preparadas (ORM) o procedimientos almacenados (servidor). - Validación y sanitización de entrada (servidor). - Principio de mínimo privilegio (servidor). - Escapado de caracteres especiales (servidor). - Actualización del software (servidor).
Dónde se aplican las mitigaciones	- Lado del servidor (principalmente). - Navegador (interpretación de la CSP). - WAF (firewall de aplicaciones web). - Frameworks modernos del lado cliente reducen el riesgo pero no lo eliminan completamente.	- Lado del servidor (principalmente). - Navegador (interpretación de la CSP). - WAF (firewall de aplicaciones web). - Frameworks modernos del lado cliente reducen el riesgo pero no lo eliminan completamente.	- Lado del servidor (principalmente). - WAF (firewall de aplicaciones web). - En el cliente NO se puede mitigar.

Recuerda que en ambos casos la mitigación principal se hace en el lado servidor: SENTENCIAS PREPARADAS para INYECTION SQL y Validación/sanitización de entrada y salida para XSS. En el caso de XSS los frameworks modernos del lado cliente reducen mucho la vulnerabilidad XSS pero no del todo (ver https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html parte framework) y en el caso de inyección SQL en el cliente no se puede mitigar nada.

A los ataques XSS se les llama ataques del lado cliente y a los Inyección SQL del lado servidor. CSP se establece en el servidor (desde donde se mandan esas cabeceras) y el navegador las interpreta

🔗 ¿Qué cambia entre ambos XSS?
En el **reflejado**, el código malicioso se **envía en petición URL** y se ejecuta por que el servidor lo devuelve en la respuesta tal como lo recibe.
En el **almacenado**, se **envía primero**, el servidor lo **guarda**, y luego lo devuelve en la respuesta y se ejecuta (cuando alguien carga la página que contiene ese dato, comentarios, blogs, chats).

CSP es una **cabecera HTTP** que el **servidor envía al navegador** para que solo ejecute contenido de fuentes seguras y no de dominios externos:
Content-Security-Policy: script-src 'self'

Los **tokens CSRF** (Cross-Site Request Forgery tokens) se usan para **proteger aplicaciones web contra ataques CSRF**, también conocidos como **falsificación de peticiones entre sitios**.

♦ ¿Qué es un ataque CSRF? Atacante **engaña usuario autenticado** para que ejecute **una acción no deseada** en una aplicación web en la que ya ha iniciado sesión. Ejemplo: Estás logueado en tu banco. Visitas sin darte cuenta una web maliciosa. Esta lanza una petición para transferir dinero desde tu cuenta sin que tú lo sepas.
♦ ¿Qué hacen los tokens CSRF? Cuando el servidor genera un formulario, añade un **token único y secreto** como campo oculto. Ese token se envía al cliente (usuario). Cuando el usuario envía el formulario, el servidor **verifica que el token coincide** con el que esperaba.

🔑 ¿Qué logran? Si la petición **viene del usuario legítimo**, el token es correcto ✅
Si la petición **es generada desde otro sitio** (como una web maliciosa), el token está ausente o incorrecto ❌ → Por tanto, **el ataque falla**.
♦ ¿Dónde se usan? Formularios HTML (cambiar contraseña, borrar cuenta, hacer transferencias...) Peticiones POST sensibles Paneles de administración

🌸 TABLA RESUMEN COMPLETA (en horizontal)

🔍 Tipo de ataque (-a)	🔢 Tipo de hash (-m)	🔤 Máscaras (?)
0 Diccionario	0 → MD5	?d → Dígito (0-9)
1 Combinación diccionarios	1000 → NTLM (Windows)	?l → Letra minúscula (a-z)
3 Máscara	5600 → NetNTLMv2	?u → Letra mayúscula (A-Z)
6 Diccionario + máscara	13100 → Kerberos TGS-REP	?s → Símbolos (!@#\$)
7 Máscara + diccionario	22000 → WPA/WPA2/WPA3	?a → Cualquier carácter
	3200 → bcrypt	?h → Hex minúscula (0-9, a-f)
	100 → SHA1	?H → Hex mayúscula (0-9, A-F)

🔑 Crackear un hash MD5 con diccionario:
echo "8743b52063cd84097a65d1633f5c74f5" > hash.txt
hashcat -m 0 -a 0 hash.txt /usr/share/wordlists/rockyou.txt

🔑 Crackear NetNTLMv2 con diccionario:
echo "admin::N46iSNekpT:..." > hash.txt
hashcat -m 5600 -a 0 hash.txt /usr/share/wordlists/rockyou.txt

🔑 Combinación de diccionarios (-a 1) + MD5 (-m 0)

🔑 Crackear NTLM con una máscara tipo PIN de 4 dígitos:
echo "b4b9b02e6f09a9bd760f388b67351e2b" > hash.txt
hashcat -m 1000 -a 3 hash.txt ?d?d?d?d

🔑 Diccionario (-a 0) + MD5 (-m 0)
echo "8743b52063cd84097a65d1633f5c74f5" > hash.txt
hashcat -m 0 -a 0 hash.txt /usr/share/wordlists/rockyou.txt

🔑 Diccionario + máscara (-a 6) + NTLM (-m 1000)

```
echo "098f6bcd4621d373cade4e832627b4f6" > hash.txt
hashcat -m 0 -a 1 hash.txt dict1.txt dict2.txt
# Ejemplo: une "admin" + "123" → admin123
```

```
echo "b4b9b02e6f09a9bd760f388b67351e2b" > hash.txt
hashcat -m 1000 -a 6 hash.txt /usr/share/wordlists/rockyou.txt ?d?d
# Ejemplo: prueba "password00" a "password99"
```

DOKER: Conceptos Básicos y Comandos Esenciales

¿Qué es Docker?

Docker es una plataforma de virtualización de contenedores que permite empaquetar una aplicación y sus dependencias en un contenedor aislado. Esto garantiza que la aplicación se ejecute de manera consistente en cualquier entorno.

Conceptos Clave:

Imagen (Image): Una plantilla de solo lectura con instrucciones para crear un contenedor Docker.

Contenedor (Container): Una instancia en ejecución de una imagen.

Volumen (Volume): Un mecanismo para persistir datos generados por un contenedor.

Red (Network): Aísla y conecta contenedores.

Comandos Básicos:

`docker pull <imagen>`: Descarga una imagen desde Docker Hub.

`docker images`: Lista las imágenes disponibles localmente.

`docker run <imagen>`: Ejecuta un contenedor desde una imagen.

`docker run -d <imagen>`: Ejecuta el contenedor en segundo plano (modo "detached").

`docker run -p <host_port>:<container_port> <imagen>`: Publica un puerto del contenedor en el host.

`docker run -v <host_path>:<container_path> <imagen>`: Monta un volumen del host en el contenedor.

`docker run -e <VARIABLE=valor> <imagen>`: Define variables de entorno dentro del contenedor.

`docker ps`: Lista los contenedores en ejecución.

`docker ps -a`: Lista todos los contenedores (en ejecución y detenidos).

`docker stop <contenedor>`: Detiene un contenedor en ejecución.

`docker rm <contenedor>`: Elimina un contenedor detenido.

`docker rmi <imagen>`: Elimina una imagen local.

Dockerfile: Creación de Imágenes Personalizadas

¿Qué es Dockerfile?

Un archivo de texto con instrucciones para construir una imagen Docker.

Instrucciones Esenciales:

`FROM <imagen_base>`: Especifica la imagen base.

`COPY <origen> <destino>`: Copia archivos desde el host al contenedor.

`RUN <comando>`: Ejecuta un comando dentro del contenedor durante la construcción de la imagen.

`CMD <comando>`: Especifica el comando que se ejecuta cuando se inicia el contenedor.

`EXPOSE <puerto>`: Indica qué puerto expone el contenedor.

`WORKDIR <directorio>`: Establece el directorio de trabajo dentro del contenedor.

`ENV <VARIABLE> <valor>`: Define variables de entorno dentro del contenedor.

Construcción de una Imagen:

`docker build -t <nombre_imagen> <directorio_Dockerfile>`: Construye una imagen a partir de un Dockerfile.

Docker Compose: Orquestación de Contenedores Multi-Contenedor

¿Qué es Docker Compose?

Una herramienta para definir y ejecutar aplicaciones multi-contenedor.

docker-compose.yml:

Un archivo YAML que define los servicios, redes y volúmenes de la aplicación.

Comandos Básicos:

`docker-compose up`: Construye e inicia los contenedores definidos en el archivo `docker-compose.yml`.

`docker-compose up -d`: Lo hace en modo "detached", es decir, en segundo plano.

`docker-compose down`: Detiene y elimina los contenedores y redes definidos en el archivo `docker-compose.yml`.

`docker-compose ps`: Lista los contenedores definidos en el archivo `docker-compose.yml`.

`docker-compose logs <service>`: Muestra los logs de un servicio concreto definido en `docker-compose.yml`.

Ventajas:

Simplifica la gestión de aplicaciones complejas.

Permite definir la configuración de la aplicación en un solo archivo.

Facilita la replicación de entornos.

Comandos de Docker:

`docker pull mysql:5.7.28`: Descarga la imagen de Docker para MySQL versión 5.7.28 desde el repositorio de Docker Hub.

`docker images`: Lista todas las imágenes de Docker descargadas localmente en tu sistema.

`docker run --rm --user mysql -d -p 3306:3306 --name mysql-db1 -e MYSQL_ROOT_PASSWORD=secret -v /mysql:/var/lib/mysql mysql:5.7.28`:

Inicia un contenedor de Docker con la imagen de MySQL 5.7.28.

`--rm`: Elimina el contenedor automáticamente cuando se detiene.

`--user mysql`: Ejecuta el contenedor con el usuario mysql.

`-d`: Ejecuta el contenedor en segundo plano (modo "detached").

`-p 3306:3306`: Mapea el puerto 3306 del contenedor al puerto 3306 del host.

`--name mysql-db1`: Asigna el nombre mysql-db1 al contenedor.

`-e MYSQL_ROOT_PASSWORD=secret`: Establece la variable de entorno MYSQL_ROOT_PASSWORD con el valor "secret".

`-v /mysql:/var/lib/mysql`: Mapea el directorio /mysql del host al directorio /var/lib/mysql del contenedor.

`docker ps -a`: Muestra una lista de todos los contenedores de Docker en tu sistema, incluyendo los que están detenidos.

`docker logs mysql-db1`: Muestra los registros (logs) del contenedor mysql-db1, lo que te permite ver la salida generada por el contenedor.

`docker exec -it mysql-db1 /bin/sh`: Inicia una sesión interactiva en el contenedor mysql-db1 usando /bin/sh, lo que te permite ejecutar comandos dentro del contenedor.

`docker stop mysql-db1`: Detiene el contenedor mysql-db1, finalizando su ejecución.

`docker start mysql-db1`: Inicia nuevamente el contenedor mysql-db1, si estaba detenido.

`docker rm -f mysql-db1`: Fuerza la eliminación del contenedor mysql-db1, incluso si está en ejecución.

`docker pull jenkins/jenkins:its`: Descarga la imagen de Jenkins.

`docker run -d -p 8080:8080 -p 50000:50000 --name jenkins-container -v jenkins_home:/var/jenkins_home jenkins/jenkins:its`: Lanza el contenedor de Jenkins.

`docker logs jenkins-container`: Muestra el log del contenedor de Jenkins.

Comandos de Dockerfile:

`FROM ubuntu`: Establece la imagen base desde la cual se construirá el contenedor. En este caso, se usa una imagen de Ubuntu.

`LABEL maintainer="admin@ejemplo.com"`: Añade una etiqueta que proporciona información sobre el mantenedor de la imagen.

`ARG APP_DATO1=1`: Declara una variable de construcción llamada APP_DATO1 con un valor por defecto de 1.

`RUN apt-get update && apt-get install -y apache2`: Ejecuta comandos en la imagen para actualizar la lista de paquetes e instalar el servidor web Apache.

`EXPOSE 80`: Indica que el contenedor escuchará en el puerto 80 en tiempo de ejecución.

`ADD index2.html /var/www/html/`: Añade el archivo index2.html desde el sistema de archivos del host al contenedor en la ruta especificada.

`ENTRYPOINT ["/usr/sbin/apache2ctl", "-D", "FOREGROUND"]`: Define el comando que se ejecutará cuando el contenedor se inicie, en este caso, se ejecuta

Apache en primer plano.

docker build -t nombre-imagen:versión: Construye una imagen a partir de un Dockerfile.

docker run -d -p 7777:80 mi-apache:1.0: Lanza un contenedor con la imagen creada a partir del Dockerfile en el puerto 7777.

-d: Ejecuta el contenedor en segundo plano (modo "detached").

-p 7777:80: Mapea el puerto 80 del contenedor al puerto 7777 de tu máquina host.

mi-apache:1.0: Especifica la imagen y la versión que acabas de crear.

Comandos y opciones de Docker Compose:

docker-compose up: Inicia y ejecuta todos los contenedores definidos en el archivo docker-compose.yml. Si es la primera vez que se ejecuta, también construirá las imágenes necesarias.

docker-compose down: Detiene y elimina todos los contenedores, redes, volúmenes e imágenes creados por docker-compose up.

services: Define los servicios (contenedores) que se ejecutarán. Cada servicio representa una imagen de Docker y puede incluir configuraciones adicionales como puertos, volúmenes y variables de entorno.

image: Especifica la imagen de Docker que se utilizará para el servicio. Puede ser una imagen pública de Docker Hub o una imagen personalizada.

build: Define la ruta del archivo Dockerfile para construir una imagen personalizada. En lugar de usar una imagen preexistente, se construye una nueva basada en las instrucciones del Dockerfile.

container-name: Especifica un nombre personalizado para el contenedor, lo que facilita su identificación y gestión.

ports: Mapea puertos entre el contenedor y el host. El formato es host:contenedor, donde el puerto del contenedor se expone al puerto especificado en el host.

volumes: Define los volúmenes que se montarán en el contenedor, permitiendo compartir datos entre el contenedor y el host. El formato es host:contenedor, donde la ruta del host se monta en la ruta especificada en el contenedor.

environment: Especifica las variables de entorno que se pasarán al contenedor. Estas variables pueden ser utilizadas por la aplicación que se ejecuta dentro del contenedor.

Comandos directamente del documento:

docker run -d -p 33060:3306 --name mysql-db1 -e MYSQL_ROOT_PASSWORD=secret -v mysql:/var/lib/mysql mysql:5.7.28

docker run -d --rm --name phpmyadminc --link mysql-db1 -e PMA_HOST=mysql-db1 -p 9080:80 phpmyadmin/phpmyadmin

Y la explicación de cada uno:

El primer comando levanta el contenedor de MySQL.

-d: Ejecuta el contenedor en segundo plano.

-p 33060:3306: Mapea el puerto 3306 del contenedor al puerto 3306 del host.

--name mysql-db1: Asigna el nombre "mysql-db1" al contenedor.

-e MYSQL_ROOT_PASSWORD=secret: Establece la contraseña de root para MySQL.

-v mysql:/var/lib/mysql: Crea un volumen llamado "mysql" y lo monta en el directorio de datos de MySQL en el contenedor.

mysql:5.7.28: Especifica la imagen de MySQL a usar.

El segundo comando levanta el contenedor de phpMyAdmin.

-d: Ejecuta el contenedor en segundo plano.

--rm: Elimina el contenedor cuando se detiene.

--name phpmyadminc: Asigna el nombre "phpmyadminc" al contenedor.

--link mysql-db1: Enlaza este contenedor al contenedor MySQL ("mysql-db1").

-e PMA_HOST=mysql-db1: Configura phpMyAdmin para que se conecte al host "mysql-db1" (el contenedor de MySQL).

-p 9080:80: Mapea el puerto 80 del contenedor al puerto 9080 del host.

phpmyadmin/phpmyadmin: Especifica la imagen de phpMyAdmin a usar.

1.¿Qué ventajas aporta trabajar con contenedores?

Portabilidad: Los contenedores encapsulan todas las dependencias necesarias para ejecutar una aplicación, lo que permite moverlos fácilmente entre diferentes entornos sin preocuparse por las diferencias en la configuración del sistema.

Escalabilidad: Los contenedores pueden escalarse rápidamente para satisfacer la demanda, ideal para aplicaciones que manejan grandes volúmenes de tráfico.

Aislamiento: Cada contenedor opera de manera independiente, los problemas en un contenedor no afectarán a otros. Esto mejora la estabilidad y seguridad del sistema.

Eficiencia de recursos: A diferencia de las MV, los contenedores comparten el núcleo del SO, lo que reduce el uso de recursos y permite un inicio más rápido.

Consistencia: Garantiza que el entorno de desarrollo, pruebas y producción sean idénticos, eliminando problemas relacionados con diferencias en la configuración.

Código del Pipeline de Jenkins

```
pipeline {
  agent any
  tools { maven 'M3' }
  stages {
    stage('Checkout') { steps { git branch: 'main', url: 'https://github.com/tu-usuario/tu-repositorio.git', credentialsId: 'github-token'; echo 'Repositorio clonado exitosamente!' } }
    stage('Build') { steps { sh 'mvn clean install'; echo 'Proyecto construido exitosamente!' } }
    stage('Test') { steps { sh 'mvn test'; echo 'Pruebas ejecutadas exitosamente!' } }
    stage('Results') { steps { junit '**/target/surefire-reports/*.xml'; archiveArtifacts '**/target/*.jar'; echo 'Resultados publicados y artefactos archivados!' } }
  }
  post {
    success { echo 'Pipeline ejecutado con éxito!' }
    failure { echo 'Pipeline falló!' }
  }
}
```

2.Revisa el pipeline que has creado en el apartado anterior ¿Qué hace?

Control de versiones: Utiliza Git para gestionar y controlar las versiones del código, permitiendo trabajo colaborativo y recuperación ante errores.

Integración continua: Automatiza la integración de cambios en el código, ejecutando pruebas unitarias y de integración para detectar errores tempranamente.

Construcción automatizada: Utiliza Jenkins para automatizar construcción de artefactos de lanzamiento, garantiza que código esté siempre listo para ser desplegado.

Pruebas automatizadas: Ejecuta pruebas unitarias, de integración y de aceptación para asegurar que el software cumple con los requisitos y no introduce regresiones.

Despliegue continuo: Automatiza el despliegue del software en entornos de prueba y producción, asegurando una entrega rápida y eficiente.

Monitorización: Recopila y analiza datos del software en producción para identificar y solucionar problemas de manera proactiva.

Explicación del Código del Pipeline con seguridad implementada:

```
pipeline { agent any } // Define un pipeline declarativo que puede ejecutarse en cualquier agente disponible de Jenkins
stages { ... } // Contiene las diferentes etapas del pipeline
```

```
stage('Checkout') // Etapa de obtención del código
```

```
steps { ... } // Define los pasos a ejecutar en esta etapa
```

```
git 'https://github.com/tu_usuario/tu_repositorio.git' // Clona el código del repositorio de GitHub
```

```
stage('Build') // Etapa de compilación
```

```
steps { ... } // Define los pasos a ejecutar en esta etapa
sh 'mvn clean install' // Compila el proyecto utilizando Maven

stage('SAST y SCA') // Etapa de análisis de seguridad estática y composición de software
steps { ... } // Define los pasos para el Análisis de Seguridad Estática (SAST) y Composición (SCA)
withSonarQubeEnv('SonarQube') { sh 'mvn sonar:sonar' } // Ejecuta el análisis de código estático con SonarQube
sh 'mvn dependency-check:check' // Ejecuta el análisis de dependencias con OWASP Dependency-Check para detectar vulnerabilidades

stage('Seguridad de Contenedor y API') // Etapa de análisis de seguridad en contenedores y API
steps { ... } // Define los pasos para la seguridad del contenedor y la API
sh 'trivy image tu_imagen:ultima' // Escanea la imagen de Docker en busca de vulnerabilidades utilizando Trivy
sh 'zap-cli -t http://tu_api --active' // Realiza pruebas de seguridad en la API utilizando OWASP ZAP

stage('Detección de Secretos') // Etapa de búsqueda de secretos en el código
steps { ... } // Define el paso para la detección de secretos
sh 'trufflehog --directory .' // Busca secretos (como contraseñas o claves API) expuestos en el código fuente con Trufflehog

stage('Pruebas y Resultados') // Etapa de pruebas unitarias y publicación de resultados
steps { ... } // Define los pasos para ejecutar las pruebas y publicar los resultados
sh 'mvn test' // Ejecuta las pruebas unitarias del proyecto
archiveArtifacts 'target/*.jar' // Guarda los archivos .jar generados en la etapa de compilación

post { ... } // Define acciones a realizar después de la ejecución de la etapa
always { ... } // Define acciones que se ejecutarán siempre, independientemente del resultado de la etapa
publishHTML ... // Publica el reporte de cobertura de las pruebas (generado por JaCoCo)
```

Análisis de código estático (SAST): Con herramientas como SonarQube o SpotBugs para escanear el código Java en busca de vulnerabilidades.
Análisis de dependencias (SCA): Usa OWASP Dependency-Check para escanear las dependencias de Maven en busca de vulnerabilidades conocidas.
Escaneo de imágenes de Docker: Con Trivy o Clair para escanear imágenes de Docker en busca de vulnerabilidades.
Pruebas de seguridad en APIs: Usa OWASP ZAP o Postman para probar la seguridad de las APIs.
Detección de secretos: Usa TruffleHog o GitGuardian para buscar secretos expuestos en el código.
Pruebas de seguridad en el build de Maven: Asegúrnos de que el build de Maven no incluya dependencias vulnerables.

Servicios Cloud:

Tipo de Servicio	Gestión del Proveedor	Gestión del Cliente	Responsabilidades sobre los Datos	Ejemplos de Responsabilidades
IaaS	Infraestructura física, servidores, almacenamiento, redes. La ubicación física de los datos en sus centros de datos.	Sistemas operativos, middleware, aplicaciones, datos. Asegurar la gestión y conformidad con las normativas locales (como GDPR).	El cliente es totalmente responsable de la seguridad y conformidad de los datos.	Configurar redes, seguridad, proteger y respaldar datos, decidir dónde se almacenan.
PaaS	Infraestructura, sistemas operativos, middleware. Ubicación de los datos en su plataforma gestionada.	Aplicaciones y datos. Garantizar que los datos estén seguros y conformes a la legislación aplicable.	El cliente es el principal responsable de los datos , pero el proveedor asegura la infraestructura de soporte.	Desarrollo de aplicaciones, configuración personalizada, cumplimiento con normativas de datos.
SaaS	Infraestructura, sistemas operativos, middleware, aplicaciones. Define la ubicación de los datos (aunque es limitada para el cliente).	Configuración de usuarios finales, gestión de acceso y uso, y asegurarse de respetar las leyes de protección de datos.	El cliente gestiona el acceso y uso de los datos ; el proveedor asegura la infraestructura y aplica medidas de seguridad básicas.	Gestionar acceso de usuarios, supervisar la seguridad de los datos y la privacidad.

La enumeración DNS es una técnica utilizada en ciberseguridad para recopilar información sobre un dominio o una red a través del sistema de nombres de dominio (DNS). Esta técnica puede proporcionar datos valiosos que pueden ser utilizados tanto para fines defensivos como ofensivos. A continuación, se detalla qué tipo de información se puede extraer con la enumeración DNS:

- 1. Información sobre Dominios y Subdominios
Lista de subdominios: Identificar subdominios asociados a un dominio principal (por ejemplo, mail.ejemplo.com, api.ejemplo.com).
Dominios relacionados: Descubrir otros dominios que puedan estar asociados a la misma organización.
- 2. Registros DNS
Registros A: Direcciones IP asociadas a un dominio o subdominio.
Registros MX: Servidores de correo electrónico utilizados por el dominio.
Registros CNAME: Aliás o nombres canónicos que redirigen a otros dominios.
Registros TXT: Información adicional, como políticas de seguridad (SPF, DKIM, DMARC) o verificaciones de propiedad.
Registros NS: Servidores de nombres que gestionan el DNS del dominio.
Registros SOA: Información sobre la autoridad del dominio, como el servidor primario y tiempos de actualización.
Registros PTR: Resolución inversa de IP a nombre de dominio.
- 3. Información sobre Infraestructura
Servidores expuestos: Identificar servidores web, de correo, FTP, u otros servicios asociados al dominio.
Balanceadores de carga: Detectar si se utilizan balanceadores de carga o CDN (Content Delivery Networks).
Direcciones IP: Obtener las direcciones IP de los servidores asociados al dominio.
- 4. Información sobre Configuraciones de Seguridad
Políticas de correo: Extraer registros TXT para verificar configuraciones de seguridad como SPF, DKIM o DMARC.
Zonas DNS mal configuradas: Identificar configuraciones incorrectas que puedan ser explotadas (por ejemplo, transferencias de zona no restringidas).
- 5. Información sobre Terceros
Proveedores externos: Identificar servicios de terceros utilizados por la organización (por ejemplo, servicios en la nube, CDN, proveedores de correo).
Dominios externos asociados: Descubrir dominios que puedan estar relacionados con la organización pero no sean directamente visibles.

6. Información Geográfica

Ubicación de servidores: A través de las direcciones IP, se puede inferir la ubicación geográfica de los servidores.

7. Información para Ataques

Puntos de entrada: Identificar servidores o servicios expuestos que puedan ser vulnerables.

Subdominios olvidados: Descubrir subdominios que puedan estar mal configurados o desactualizados.

Ampliación de superficie de ataque: Obtener una lista completa de activos asociados al dominio.

Herramientas Comunes para Enumeración DNS

nslookup: Consulta básica de registros DNS.

dig: Herramienta avanzada para consultar registros DNS.

Whois: Obtener información sobre el registro del dominio.

Sublist3r: Enumeración de subdominios.

DNSdumpster: Herramienta en línea para enumeración DNS.

Fierce: Escaneo de dominios y subdominios.

Amass: Herramienta avanzada para mapeo de dominios y subdominios.

Uso de la Información

Defensivo: Mejorar la seguridad de la infraestructura, identificar configuraciones incorrectas y proteger activos expuestos.

Ofensivo: Identificar posibles vectores de ataque, como servidores vulnerables o subdominios olvidados.

NMAP